

AORRTC：几乎必然渐进最优的 RRT-Connect 规划

Tyler S. Wilson¹, Wil Thomason², Zachary Kingston³, and Jonathan D. Gammell¹

Abstract—在运动规划中快速找到高质量的解决方案是一个重要目标。对于高自由度的机器人尤其如此。传统的满意规划算法通常能够快速找到可行的解决方案，但不能保证其最优性，而最优规划算法在收敛到最优解方面具有概率保证，但计算开销较大。本文使用 AO- x 元算法将满意的 RRT-Connect 规划器扩展到最优规划。结果得出的 Asymptotically Optimal RRT-Connect (AORRTC) 在与 RRT-Connect 相似的时间内找到初始解决方案，并利用任何额外的规划时间以随时的方式趋近于最优解。它被证明为概率完全且 almost-surely asymptotically optimal (a.s.a.o.)。AORRTC 在 MotionBenchMaker 数据集上测试了 Panda (7 自由度) 和 Fetch (8 自由度) 机器人臂。这些实验显示，AORRTC 找到初始解决方案的速度与 RRT-Connect 一样快，并且比测试的最新 a.s.a.o. 算法更快地收敛到更好的解决方案。AORRTC 在几毫秒内找到困难的高自由度规划问题的解决方案，而其他 a.s.a.o. 规划器在几秒内不能一致地找到解决方案。这种性能在使用和不使用 single instruction/multiple data (SIMD) 加速时都得到了证明。

I. 引言

运动规划旨在快速找到给定问题的高质量解决方案，特别是在规划高 degree-of-freedom (DoF) 机器人或实时规划时。运动规划算法在机器人的连续配置空间空间 (i.e., 搜索空间) 的离散近似中进行搜索。

运动规划算法以不同方式近似搜索空间。基于图的规划器，例如 Dijkstra 算法 [dijkstra] 和 A* [astar]，需要事先对搜索空间进行离散化。高分辨率的近似通常包含高质量的解决方案，但计算上昂贵，而低分辨率的近似则较便宜，但可能仅包含低质量的解决方案，甚至根本没有解决方案。轨迹优化方法，例如 CHOMP [chomp] 和 TrajOpt [trajopt]，对其近似的依赖性较小，但仅提供局部保证，在解决困难规划问题时可能会陷入局部极小值而找不到解决方案 [trajopt]。

基于采样的规划器，例如 Probabilistic Roadmaps (PRM) [prm] 和 Rapidly-exploring Random Trees (RRT) [rrt]，通过逐步采样搜索空间，避免了需要进行先验近似。这使它们能够搜索底层连续空间的越来越精确的近似，并使其成为高维问题的流行且有效的选择。基于采样的规划器提供概率保证。算法被称为概率完备的，如果其找到解决方案的概率在样本数量趋于无穷大时趋于 1，前提是存在解决方案。如果它们在无限数量的样本情况下以概率 1 渐进收敛于最优解，则称为 almost-surely asymptotically optimal (a.s.a.o.) [prmstar]。

RRT-Connect [rrtc] 扩展了 RRT，以交替搜索从起始点到目标点的可行路径。这个双向满意规划器因其

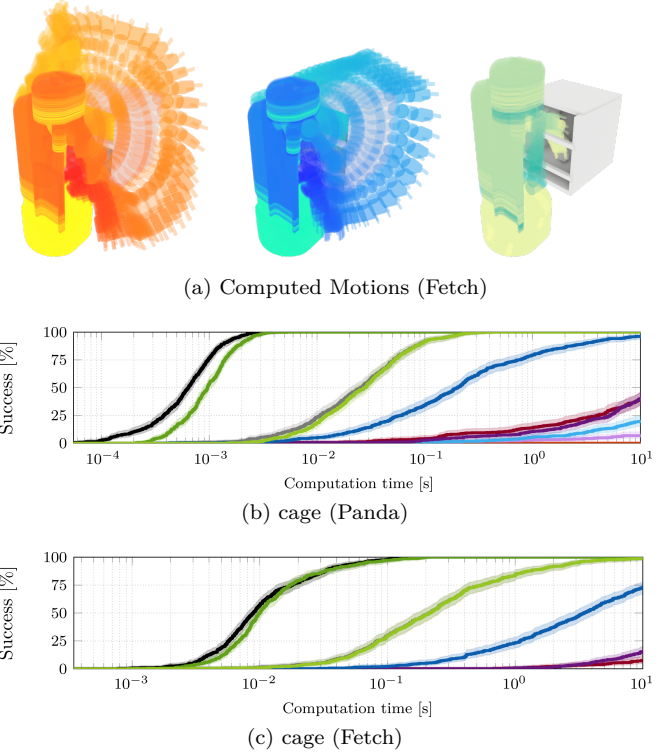


Fig. 1: 来自 MotionBenchMaker [mbm] 数据集的所有规划器在 7 自由度 Panda 问题中的结果，以及一部分规划器在 8 自由度 Fetch 问题中的结果。在具有挑战性的 8 自由度 Fetch 笼子问题 (Fig. 3c) 中，路径显示在 (a) 中，VAMP RRT-Connect 在 62 毫秒后找到的解决方案为橙色，VAMP AORRTC (i.e., RRT-Connect 加路径简化) 在 63 毫秒后找到的初始解决方案为蓝色，VAMP AORRTC 在 3.5 秒后找到的最后解决方案为绿色。在笼子环境中于给定时间内找到解决方案的试验百分比显示在 (b) 和 (c) 中，具有 Clopper-Pearson 99 % 置信区间 [pdt]。VAMP 和 OMPL AORRTC 的初始解决方案找到的速度明显快于所有其他测试的 VAMP 和 OMPL a.s.a.o. 规划器。

简单的实现和快速的初始解时间而被广泛使用。RRT-Connect 不提供解的质量保证，并且不会通过更多的采样来改善其解 (i.e., 它不是 a.s.a.o.)。路径平滑或简化 [shortcut1, shortcut2, bspline, hybrid] 可以改善 RRT-Connect 找到的通常较低质量解的成本 (Fig. 1a)，但不提供全局质量保证。

诸如 a.s.a.o. 的即时规划器，例如 RRT* [prmstar] 和 Batch Informed Trees (BIT*) [bit]，在概率上保证能找到一个解决方案，然后收敛到最优解。这些算法利用额外的规划时间来改善当前的解决方案，但为保证几乎确定的渐进最优性所需的开销可能会增加找到初始解决方案的时间。研究集中在如何改善初始解决方案时间和提升向最优解收敛的速度 [ait]。

¹Estimation, Search, and Planning (ESP) Research Group, Queen's University, Kingston ON, Canada. {18tsw1,gammell}@queensu.ca

²wil.thomason@gmail.com

³Department of Computer Science, Purdue University, West Lafayette IN, USA zkingston@purdue.edu

AO- x 元算法 [a ox :1] 提出了一个对最优规划问题的替代性表述。与其要求规划器在配置空间中优化一条可行路径的成本, AO- x 要求规划器在一个已扩展以包含额外维度的搜索空间中找到一条可行路径。这个额外的维度表示到达每个配置的成本。对这些扩展的搜索空间序列调用满足规划器,并在成本维度上适当地降低限制,被证明是 a.s.a.o. [a ox :1, a ox :2]。

Asymptotically Optimal RRT-Connect (AORRTC) ¹ 应用了 AO- x 和 RRT-Connect 的思想,在一个成本增强的搜索空间中执行双向搜索。这使得初始解的查找速度和 RRT-Connect 一样快,并且朝着高质量解的收敛速度比其他 a.s.a.o. 算法快多个数量级。AORRTC 的性能通过 Open Motion Planning Library (OMPL) [ompl] 和 Vector-Accelerated Motion Planning (VAMP) [vamp] 实现进行了测试。这些测试展示了该方法在有和无 single instruction/multiple data (SIMD) 加速情况下的有效性,特别是表明了 AORRTC 在有 SIMD 加速的情况下,可以在微秒内逼近高维规划问题的最优解。两种 AORRTC 的实现都在 MotionBenchMaker (MBM) 数据集中,针对 7 自由度的 Panda 和 8 自由度的 Fetch 规划问题进行了评估。AORRTC 找到初始解的速度与 RRT-Connect 一样快,且比所有测试的 a.s.a.o. 规划器都快,并且可以一致地解决更多问题 (Figs. 1 and 2)。它也比测试的 a.s.a.o. 规划器更快地收敛到更好的解。这些相对性能在 AORRTC 的 OMPL 和 VAMP 实现中均得到了保持 (Sec. IV)。

II. 相关工作

提高实际运动规划的性能通常着重于在更短的时间内找到更高质量的解决方案。已经有大量工作在考虑这些目标的情况下改进基于采样的运动规划。

A. 几乎必然渐近最优规划

任何时刻, a.s.a.o. 规划器利用额外的规划时间来改进它们对搜索空间的近似并找到更好的解决方案。这些规划器渐近地以概率为一 (i.e., 几乎肯定) 收敛到最优解。

最近邻查找和边缘评估是基于采样的运动规划中主要的计算成本 [bottleneck]。这些计算开销大的操作在 a.s.a.o. 规划器中通常执行得更频繁,以保证几乎确定的渐近最优性,这可能导致它们比满意规划器更慢找到初始解决方案。例如 RRT* [prmstar] 和 BIT* [bit] 这样的即时 a.s.a.o. 规划器,通过迭代采样搜索空间来改善其近似和解决方案。RRT* 通过增量重接新的顶点来降低路径成本,而 BIT* 按启发式确定的顺序搜索样本批次,以根据潜在解决方案质量的顺序考虑状态。这些规划器需要付出计算努力来维持其 a.s.a.o. 保证,并且大量工作集中于提高其性能。

几种 RRT* 扩展改善了规划器收敛到最优解的能力。RRT # [rrtsharp] 构建了一棵搜索树,其中包含可能属于最优解的所有顶点的最佳达到成本信息,而不仅仅是局部重连顶点。RRT*-Smart [rrt*smart] 和 Informed RRT* [informed] 则通过利用特定问题的信息来提高规划器的性能。这些规划器利用先前搜索工作的信息来更智能地生成样本,以在改善解决方案时减少规划时间,但这种知情采样无法帮助找到初始解决方案。

Fully Connected Informed Trees (FCIT*) [fcit] 利用 VAMP 的 SIMD 并行性所支持的边缘评估成本降低来搜索一个完全连接的图。这充分利用了搜索空间的近似性,并避免了维护昂贵的最近邻结构的需要。FCIT* 比非 SIMD 加速规划器 (包括 RRT-Connect) 能更快地找到高质量的解决方案,但不能像 SIMD 加速的 RRT-Connect 那样快速找到解决方案。

AORRTC 类似于 RRT* 和 BIT*, 通过增加计划时间来提高其找到的解决方案的质量,但其找到初始解决方案的速度与 RRT-Connect 一样快。与 RRT* 或 RRT # 不同, AORRTC 无需重新布线其树以保持其 a.s.a.o. 保证,而是在添加新顶点时随机采样较低的成本界限,以便可能将其连接到成本较低的父节点。AORRTC 利用信息采样方式,如同在 Informed RRT* 中进行,但在找到解决方案后重新开始搜索,而不是继续使用相同的树进行计划。

B. 双向规划

双向采样规划器,例如 RRT-Connect [rrtc], 通过从起始和目标顶点同时延展树并尝试连接这些树来探索搜索空间。RRT-Connect 在大多数现实世界的操作问题上比其他规划算法更快地找到初始解决方案 [compreview], 但不保证解决方案质量,并且通常找到低质量解决方案。有些双向 a.s.a.o. 规划器,例如 Bidirectional RRT* (B-RRT*) [brtt*] 和 RRT*-Connect [rrt*connect], 通过 RRT* 的 a.s.a.o. 保障扩展了 RRT-Connect 的快速双向搜索。这些算法在整个搜索过程中需要 RRT* 风格的重连,以保证几乎肯定的渐近最优性。这种增加的计算成本相对于 RRT-Connect 增加了找到初始解决方案所需的时间。

其他双向 a.s.a.o. 规划器,如 Adaptively Informed Trees (AIT*) 和 Effort Informed Trees (EIT*) * [ait], 则使用一种不对称的双向搜索,其中信息在反向和正向搜索之间共享。轻量级的反向搜索根据当前样本的分布来计算启发式信息,以指导计算密集的正向搜索。正向搜索有效地找到解决方案,并将碰撞检查信息传递给反向搜索以更新启发式信息。这减少了边缘评估的成本,但其效用取决于边缘评估相对于最近邻查找的成本。

AORRTC 执行与 B-RRT* 和 RRT*-Connect 相似的 a.s.a.o. 双向搜索,但不需要 RRT* 风格的重连连接来保持其 a.s.a.o. 保证。与 AIT* 和 EIT* 不同, AORRTC 同时增长两棵树并尝试连接它们,而不是使用反向搜索来计算启发式。

C. 扩展搜索空间

增强的搜索空间已在其他规划器中用于改善规划。Time-based RRT (TB-RRT) [tbrtt] 增强了配置空间,通过添加时间维度来进行动态障碍物或目标的规划。这个时间增强空间将原始问题中的动态障碍物转化为时间增强搜索空间中的静态障碍物 [roadmap_dynamic], 从而简化了动态环境中的规划,但对解决时间或质量不提供任何保证。

其他规划器,如 Windowed Heirarchical Cooperative A* (WHCA*) 至 [whca] 和 Safe Interval Path Planning (SIPP) 至 [sipp], 抽象出一个时间增强的搜索空间以减少规划时间。WHCA* 在时间增强的搜索空间中规划,直到达到用户指定的阈值,然后排除时间维度和任何动态障碍物以备后用。SIPP 将连续的时间维度离散化为离

¹发音为主动脉 (ā - 'ò r-tik)。

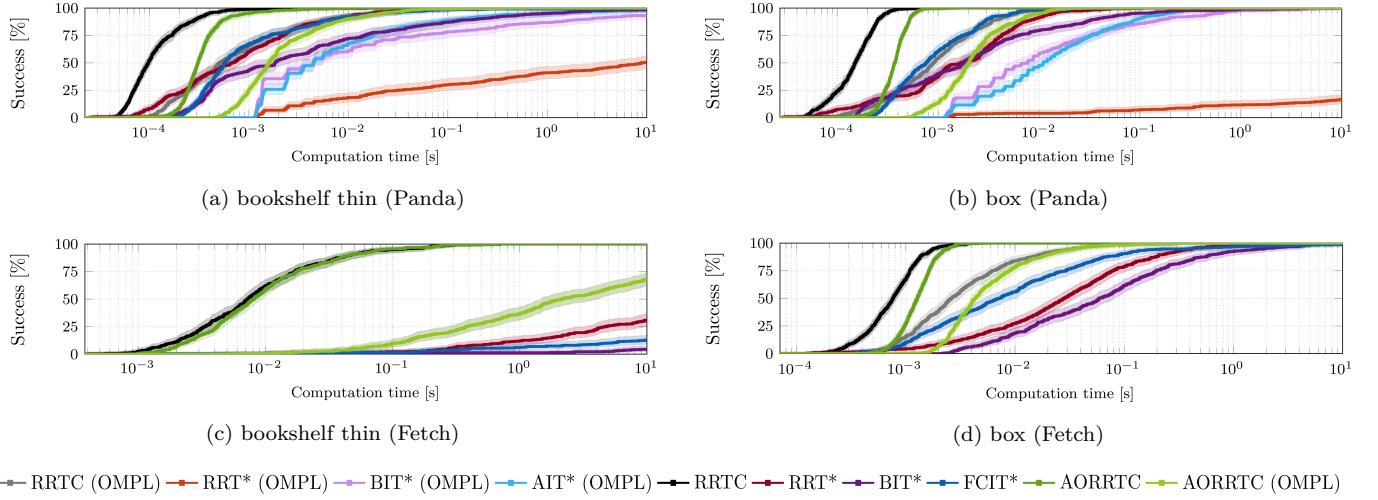


Fig. 2: 对于 7 自由度 Panda 和 8 自由度 Fetch 机器人上的所有问题，我们进行了 5 次试验，其结果包括所有规划器的初始解决方案，以及这些规划器在 MotionBenchMaker 数据集中书架薄环境 ((a) 和 (c)) 和盒子环境 ((b) 和 (d)) 中的子集。每个图表显示了在任意给定时间内找到解决方案的运行百分比，并带有 Clopper-Pearson 99% 置信区间。VAMP 比所有其他测试的规划器更快找到初始解决方案。

散的安全间隔，这些间隔描述了配置被认为有效的持续时间，并在这个简化的状态-安全间隔增强的搜索空间中规划。这些搜索空间抽象有助于提高规划器性能，但对解决方案质量不提供保证。

一些双向规划器，如 Space-Time RRT* (ST-RRT*) [strrtstar] 和 Safe-Interval RRT (SI-RRT) [sirrt]，将 RRT-Connect 扩展到时间扩展的搜索空间中。ST-RRT* 在时间维度上搜索逐步增加的范围，以迅速在无限时间空间中找到初始解决方案，但需要 RRT* 风格的重新连接来保持其对解决方案质量的保证。SI-RRT 则在一个简化的状态安全区间扩展搜索空间中搜索，以快速在高维动态环境中找到初始解决方案，但对解决方案质量不提供保证。

元算法 AO- x [aox:1] 通过使用增加了成本的搜索空间来扩展满意的运动规划器，以包括 a.s.a.o. 的保证。这个搜索空间包括配置空间的 n 维和一个描述达到每个状态的成本的 $(n+1)^{\text{th}}$ 维。当成本函数和机器人的动态是利普希茨连续时，满意的规划器几乎可以确定以渐近方式收敛到最优解，通过在这个扩展的搜索空间中找到一系列可行的计划。这个元算法已经应用于 Expansive Space Tree (EST) [aox:1] 和 RRT [aox:2]。

AORRTC 搜索一个扩展的空间，就像 TB-RRT 和 ST-RRT* 一样，但这个空间是增加了成本维而不是时间维。不同于 WHCA*、SIPP 和 SI-RRT，AORRTC 并没有搜索其扩展搜索空间的抽象。

III. ASYMPTOTICALLY OPTIMAL RRT-CONNECT

AORRTC 将 AO- x 和 RRT-Connect 的思想应用于创建一个随时可用的 a.s.a.o. 规划器，该规划器能够像 RRT-Connect 一样快速找到初始解决方案，并在附加的规划时间内以随时可用的方式渐近收敛到最优解。AORRTC 的伪代码在 Alg. 1 中展示，其中对 RRT-Connect (11-??) 的更改用红色标记。

AORRTC 在扩展搜索空间中的一系列问题上以解决方案成本的开放上限迭代运行 RRT-Connect。这个上限首先由 RRT-Connect 经过路径简化技术 (e.g., 随机化捷径 [shortcut1, shortcut2] 和 B 样条平滑 [bspline])

应用后的初始解决方案确定 (Line 2)。然后，AORRTC 使用这个上限来设置一个新的规划问题，其中扩展空间的成本维度由当前最佳解决方案的成本限制 (Line 5)。每次找到更好的解决方案时，这个上限都会降低 (7-??)。AORRTC 在扩展搜索空间中从起始和目标顶点生长树，其中每个顶点由一个配置及其在各自树中的到来成本组成 (Lines 12 and 13)。当前树朝扩展搜索空间中的一个随机样本生长，该样本由一个随机配置 (Line 15) 和一个随机采样的成本上限 (Line 16) 组成。这个采样的成本上限是连接到样本的成本上限，并将连接限制在仅能对当前最佳解决方案产生更高质量的那些连接。顶点连接到树后，其采样的成本上限会从一个较低范围内重新随机采样，以查看顶点是否可以更低成本轻松连接 (___ 结束 @cref27-??)。基础的 RRT-Connect 搜索通过强制样本只尝试满足其随机采样成本上限的连接来增量地找到更高质量的解决方案 (Line 46)。一旦找到解决方案，它会被简化并确定一个新的解决方案成本上限。随后，以这个更严格的成本上限重新开始搜索。这样限制每次后续搜索为比当前解决方案更高质量的路径，并允许规划者几乎肯定地渐近收敛到一个最优解决方案，即使其基础搜索不是 a.s.a.o.。

我们用 $X \subseteq \mathbb{R}^n$ 表示配置空间，其中无效和有效的子集分别表示为 $X_{\text{invalid}} \subseteq X$ 和 $X_{\text{valid}} := \text{closure}(X \setminus X_{\text{invalid}})$ 。设 $x \in X$ 为一个配置， $x_{\text{start}} \in X_{\text{valid}}$ 为起始配置， $x_{\text{goal}} \in X_{\text{valid}}$ 为目标配置。令 $c \in \mathbb{R}^+$ 为配置空间中一条边或连续几条边 (即 i.e., 一条路径) 的代价。我们将搜索以树的形式存储，表示为 $T := (V, E)$ ，每棵树由一组顶点 $V \subseteq \mathbb{R}^{n+1}$ 和一组边 $E \subseteq V \times V$ 组成。每条边 $e := (v_p, v_c) \in V \times V$ 连接两个顶点，我们分别称之为边的父节点和子节点。我们用边的序列 $\sigma = (x_s, v_0), (v_0, v_1), (v_1, v_2), \dots, (v_q, x_g)$ 表示一个解，其中 $(v_i, v_j) \in E$ 。

函数 $c : X \times X \rightarrow [0, \infty)$ 计算两个配置之间的边代价，或者，稍作滥用符号，计算给定解的代价。函数 $\hat{c} : X \times X \rightarrow [0, \infty)$ 是边代价的可接受估计，i.e., $\forall x_p, x_c \in X, \hat{c}(x_p, x_c) \leq c(x_p, x_c)$ 。函数 $g_T : V_T \rightarrow [0, \infty)$ 代表从根到给定顶点 v 通过树 T 的来路代价。函

Algorithm 1: AORRTC

```

1 Function aorrtc ()
2    $\sigma_0 \leftarrow \text{simplify}(\text{rrt-connect}(\infty))$ ;
3    $\sigma_{\text{best}} \leftarrow \sigma_0$ ;  $c_{\min} \leftarrow c(\sigma_0)$ ;  $\epsilon \leftarrow \epsilon_{\text{init}}$ ;
4   repeat
5      $\sigma_i \leftarrow \text{simplify}(\text{rrt-connect}(c_{\min}))$ ;
6     if  $\sigma_i \neq \emptyset$  then
7        $\sigma_{\text{best}} \leftarrow \sigma_i$ ;
8        $c_{\min} \leftarrow c(\sigma_i)$ ;
9   until stop;
10  return  $\sigma_{\text{best}}$ ;

11 Function rrt-connect ( $c_{\max}$ )
12   $V_a \leftarrow \{(x_{\text{start}}, 0)\}$ ;  $E_a \leftarrow \emptyset$ ;  $T_a := \{V_a, E_a\}$ ;
13   $V_b \leftarrow \{(x_{\text{goal}}, 0)\}$ ;  $E_b \leftarrow \emptyset$ ;  $T_b := \{V_b, E_b\}$ ;
14  repeat
15     $x_{\text{rand}} \sim \mathcal{U}(X_{\text{free}})$ ;
16     $c_{\text{rand}} \sim \mathcal{U}\left(\left(\hat{g}_{T_a}(x_{\text{rand}}), c_{\max} - \hat{h}_{T_a}(x_{\text{rand}})\right)\right)$ ;
17     $x_{\text{near}} \leftarrow \text{nearest}(T_a, x_{\text{rand}}, c_{\text{rand}})$ ;
18     $x_{\text{new}} \leftarrow \text{steer}(x_{\text{rand}}, x_{\text{near}})$ ;
19    if validate ( $x_{\text{near}}, x_{\text{new}}$ ) then
20       $c_{\text{new}} \leftarrow \text{extend}(T_a, x_{\text{near}}, x_{\text{new}})$ ;
21      if connect ( $T_b, x_{\text{new}}, c_{\text{new}}$ ) then
22        return  $\text{path}(T_a, T_b)$ ;
23    swap ( $T_a, T_b$ );
24  until timeout;
25  return  $\emptyset$ ;

26 Function extend ( $T, x_{\text{near}}, x_{\text{new}}$ )
27  repeat
28     $x_p \leftarrow x_{\text{near}}$ ;
29     $c_{\text{new}} \leftarrow g_T(x_p) + \hat{c}(x_p, x_{\text{new}})$ ;
30     $c_{\text{rand}} \sim \mathcal{U}((\hat{g}_T(x_{\text{new}}), c_{\text{new}}))$ ;
31     $x_{\text{near}} \leftarrow \text{nearest}(T, x_{\text{new}}, c_{\text{rand}})$ ;
32  until  $x_{\text{near}} \equiv x_p$  or not validate ( $x_{\text{near}}, x_{\text{new}}$ );
33   $E \stackrel{\perp}{\leftarrow} (x_p, x_{\text{new}})$ ;
34   $V \stackrel{\perp}{\leftarrow} (x_{\text{new}}, c_{\text{new}})$ ;
35  return  $c_{\text{new}}$ ;

36 Function connect ( $T_b, x, c$ )
37  repeat
38     $x_{\text{near}} \leftarrow \text{nearest}(T_b, x, c_{\max} - c)$ ;
39     $x_{\text{new}} \leftarrow \text{steer}(x, x_{\text{near}})$ ;
40    if validate ( $x_{\text{near}}, x_{\text{new}}$ ) then
41      extend ( $T_b, x_{\text{near}}, x_{\text{new}}$ );
42  until  $x_{\text{new}} \equiv x$  or not validate ( $x_{\text{near}}, x_{\text{new}}$ );
43  return validate ( $x_{\text{near}}, x_{\text{new}}$ );

44 Function nearest ( $T, x, c$ )
45   $x_{\text{near}} \leftarrow$ 
46   $\arg \min_{v \in V_T} \{ \|x - x_v\|^2 + |c - c_v|^2 \mid c_v + \hat{c}(x_v, x) < c \}$ ;
47  return  $x_{\text{near}}$ ;

```

数 $\hat{g}_T : X \rightarrow [0, \infty)$ 代表此来路代价的一个可接受估计, i.e., $\forall x \in X, \hat{g}_T(x) \leq g_T(x)$ 。函数 $\hat{h}_T : X \rightarrow [0, \infty)$ 代表从配置 x 到树的目标 T 的预估代价。注意一棵树的目标 T_a 是另一棵树的根 T_b , i.e., $\forall x \in X, \hat{h}_{T_a}(x) \equiv \hat{g}_{T_b}(x)$ 。令 A 和 B 为两个集合。符号 $\mathcal{U}(A)$ 是从集合 A 中均匀抽取样本的简写。符号 $A \stackrel{\perp}{\leftarrow} B$ 是集合复合操作 $A \leftarrow A \cup B$ 的简写。

A. 扩展搜索空间

AORRTC 在一个扩展的 $(n+1)$ 维搜索空间中搜索, 其中前 n 维是配置空间, 第 $(n+1)^{\text{th}}$ 维是达到该配置的花费。新的配置均匀地从配置空间 (Line 15) 中采样。新状态的成本上限 c_{rand} 在通过该状态的解决方案可能的最小和最大成本之间随机均匀采样, 以使该状态是可行和有用的, i.e., $\hat{g}(x) + \hat{h}(x) \leq c_{\text{rand}} < c_{\max}$ (Line 16)。这一成本是一个上限, 以使连接到配置在扩展空间中被认为是有效的, 配置的真实成本将从其父配置中计算出来。在扩展的搜索空间中, 最近邻被定义为在配置和成本上都最接近的状态 (Line 46) [aox:1, aox:2]。此距离函数意味着具有高成本上限的样本最接近具有高到达成本的顶点。这导致更短的边缘, 使有效连接更有可能, 并促进探索未探测区域, 但可能增加达到新配置的成本。为了解决这一问题, 新添加顶点的成本在添加到树中被重新采样以搜索更低成本的邻居 (_ 结束 @cref27-??)。这种重新采样继续进行, 直到找到无效边或相同的父节点。在树中新顶点的成本 v 是通过其父顶点 v_p 的到达成本, i.e., $g_T(v) = g_T(v_p) + c(v_p, v)$ 。

B. 分析

在成本函数和机器人动力学是利普希茨连续且增强搜索空间上的规划器是概率完备的情况下, AO- x 元算法已被 [aox:2] 证明为 a.s.a.o.。因此, 如果其使用的 RRT-Connect 版本是概率完备的, 那么 AORRTC 也是 a.s.a.o.。原始的 RRT-Connect 是概率完备的 [rrtc], 而在 AORRTC 中添加的成本重采样 (_ 结束 @cref27-??) 仅更改连接的成本, 而不改变树中的顶点。这保持了 RRT-Connect 的概率完备性, 因此 AORRTC 也是概率完备的, 并且对于原始规划问题是 a.s.a.o.。

C. 任意时间 RRT 连接

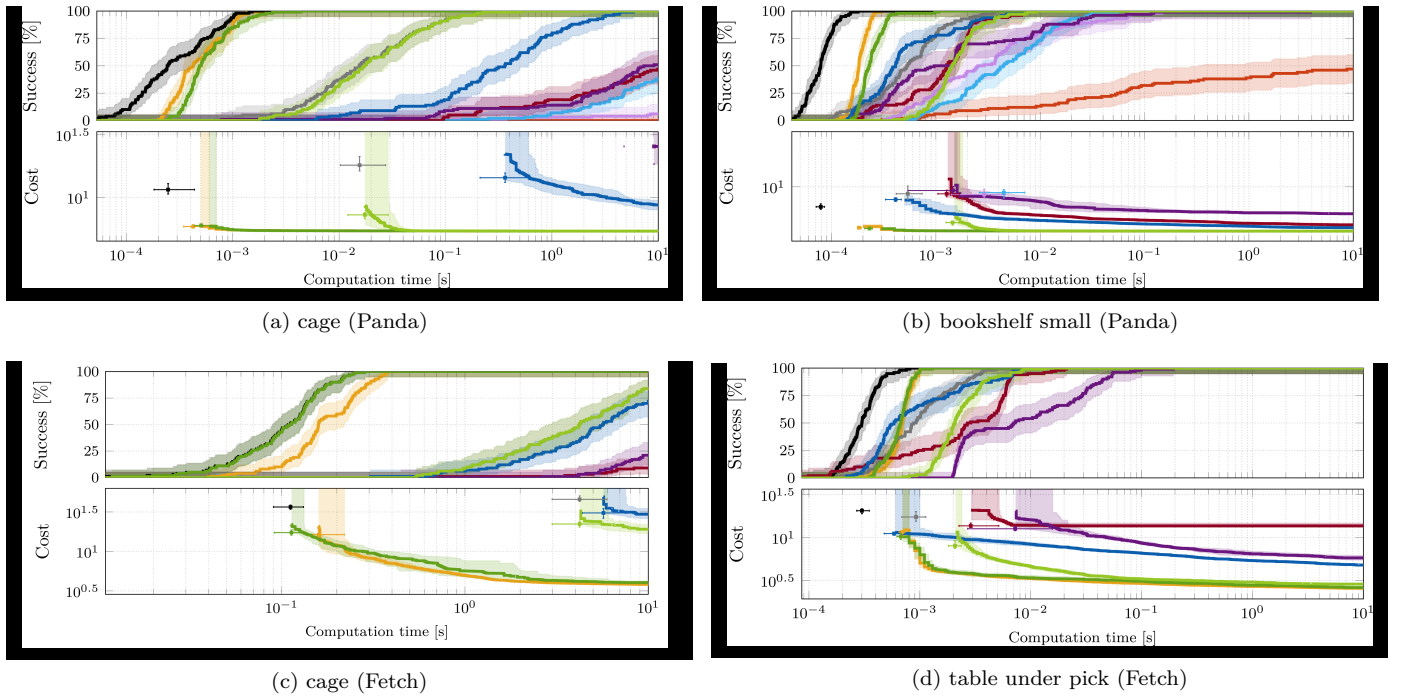
一种将 RRT-Connect 转化为一个任意时刻规划器的朴素方法是通过知情采样迭代运行它, 以尝试找到更高质量的解决方案。这种任意时刻 RRTs 的双向扩展在许多问题上收敛到一个合理的解决方案, 但可以证明它不是 a.s.a.o. [prmstar]。即使它在实际表现中没有提供正式的最优性保证, 该任意时刻 RRT-Connect 的性能展示在 Fig. 3 中。

Alg. 1 描述了 AORRTC 的一个概念版本, 这个版本为诸多实用改进留下了空间。AORRTC 应该在可用时使用知情采样 [informed] 进行配置采样, 并应用平衡的双向搜索 [balanced]。在适当的情况下, 应使用最近邻结构和查找。

对 AORRTC 的 OMPL 和 VAMP 实现与 RRT-Connect、RRT*、BIT* 和 AIT* 的 OMPL [ompl] 实现以及 RRT-Connect、RRT*、BIT* 和 FCIT* 的 VAMP [vamp] 实现进行了评估。我们还展示了在 Fig. 3 中 VAMP 实现的 Anytime RRT-Connects (??) 的结

Table I: 关于 Panda (7 自由度) 和 Fetch (8 自由度) 机器人手臂在 MotionBenchMaker [mbm] 数据集上的所有规划结果的汇总。每个结果表示在给定环境中解决问题的百分比 (加粗), 在该环境下所有问题的初始解决时间中位数, 以及在该环境下所有问题的初始路径长度中位数。

	Planner	bookshelf small	bookshelf tall	bookshelf thin	table pick	table under pick	box	cage
Panda	OMPL	RRT-Connect	100 % 0.7ms (10.3)	100 % 0.6ms (9.0) % 0.5ms (8.0) % 0.3ms (7.0) % 0.3ms (13.4)	100 % 0.9ms (10.0) % 25ms (20.8)			
		RRT*	38 % ∞ (∞)	39 % ∞ (∞) 51 % 8827ms (32.2) 6304ms (33.0) ∞ (∞)	17 % ∞ (∞) 0 % ∞ (∞)			
		BIT*	76 % 4.5ms (8.7)	91 % 3.4ms (8.3) % 4.4ms (8.3) % 1.4ms (8.3) % 2.2ms (11.4)	100 % 7.7ms (9.9) ∞ (∞)			
		AIT*	83 % 5.5ms (8.8)	95 % 4.4ms (8.3) % 4.5ms (8.3) % 2.3ms (8.3) % 3.4ms (11.5)	100 % 11.1ms (20.3) ∞ (∞)			
		AORRTC	100 % 1.8ms (5.2)	100 % 1.7ms (5.0) % 1.6ms (5.0) % 1.2ms (4.0) % 1.5ms (7.2)	100 % 2.0ms (5.0) % 28.1ms (8.3)			
	VAMP	RRT-Connect	100 % 0.1ms (7.0)	100 % 0.1ms (6.0) % 0.1ms (6.0) % 0.1ms (6.0) % 0.1ms (10.0)	100 % 0.2ms (9.0) % 0.6ms (15.4)			
		RRT*	99 % 0.7ms (8.8)	100 % 0.9ms (8.0) % 0.7ms (9.0) % 0.2ms (9.0) % 0.3ms (14.9)	100 % 1.7ms (8.0) ∞ (∞)			
		BIT*	82 % 1.7ms (9.9)	95 % 1.7ms (9.3) % 1.8ms (9.9) % 0.3ms (8.0) % 0.4ms (12.4)	100 % 1.8ms (10.5) ∞ (∞)			
		FCIT*	96 % 0.7ms (7.2)	100 % 0.5ms (7.0) % 0.6ms (7.0) % 0.3ms (6.0) % 0.5ms (10.4)	100 % 0.8ms (9.6) 201ms (14.9)			
		AORRTC	100 % 0.3ms (4.7)	100 % 0.3ms (4.0) % 0.3ms (4.0) % 0.3ms (4.0) % 0.3ms (6.4)	100 % 0.4ms (4.0) % 1.0ms (7.1)			
Fetch	OMPL	RRT-Connect	77 % 798ms (29.1)	84 % 569ms (2084) 2137ms (99.3) 1.5ms (18.0) % 2.6ms (19.5)	100 % 2.6ms (23.4) 212ms (36.5)			
		AORRTC	79 % 804ms (11.8)	84 % 570ms (1086) 2139ms (92.3) 3.1ms (9.2) % 4.1ms (9.0)	100 % 4.3ms (95.5) 214ms (17.0)			
	VAMP	RRT-Connect	100 % 5.5ms (20.1)	100 % 6.6ms (206) % 7.3ms (130) % 0.3ms (150) % 0.4ms (16.6)	100 % 0.8ms (206) % 8.9ms (29.9)			
		RRT*	36 % ∞ (∞)	36 % ∞ (∞) 31 % ∞ (∞) 99 % 6.5ms (129) % 13.3ms (12.2)	100 % 28.0ms (71.1) ∞ (∞)			
		BIT*	15 % ∞ (∞)	13 % ∞ (∞) 4 % ∞ (∞) 84 % 112ms (1828) % 368ms (14.1)	99 % 59.2ms (15.7) ∞ (∞)			
		FCIT*	27 % ∞ (∞)	29 % ∞ (∞) 13 % ∞ (∞) 94 % 4.6ms (126) % 9.3ms (12.1)	99 % 7.1ms (15.3) % 3452ms (27.8)			
		AORRTC	100 % 6.4ms (11.0)	100 % 7.1ms (108) % 8.1ms (103) % 0.8ms (8.0) % 0.8ms (9.1)	100 % 1.3ms (100) % 10ms (14.3)			



— RRTC (OMPL) — RRT* (OMPL) — BIT* (OMPL) — AIT* (OMPL) — RRTC — RRT* — BIT* — FCIT* — AORRTC — AORRTC (OMPL) — Anytime RRTCs

Fig. 3: 收敛结果显示对 7 自由度 Panda 和部分 8 自由度 Fetch 的所有规划器进行的所有 100 次试验, 针对笼子 (a) 和 (c)、小书架 (b) 以及桌子下拾取 (d) 环境中的一个问题, 数据集来自 MotionBenchMaker [mbm] (??)。每个子图的顶部显示在给定时间内找到解决方案的运行百分比, 具有 Clopper-Pearson 99 % 置信区间; 底部显示非参数 99 % 置信区间内的中位初始路径长度和随时间变化的中位路径长度 [pdt]。

果。报告的 AORRTC 的解决方案成本和时间包括随机化简化 [shortcut1, shortcut2] 和 B 样条平滑 [bspline] 的计算成本。RRT-Connect 实现的报告解决方案成本和时间不包括简化, 但这些可以从 AORRTC 的初始解决方案时间和成本推断, 因为这是一个单一简化的 RRT-Connect 搜索。RRT-Connect 和 AORRTC 都使用了平衡双向搜索 [balanced]。规划器在 MBM [mbm] 数

据集上进行测试, 该数据集包含 7 个不同机器人的规划环境, 其中每个环境有 100 个不同的规划问题²。这些环境涵盖了一系列操作规划场景, 包括到达 (高书架、小书架和薄书架)、受限的到达 (盒子和笼子) 以及桌面操作 (桌上拾取和桌下拾取)。OMPL 和 VAMP 规划器运

²由于对于 Panda 和 Fetch 机器人来说, 一些问题是无效的, 从而分别导致总共有 699 个和 679 个问题。

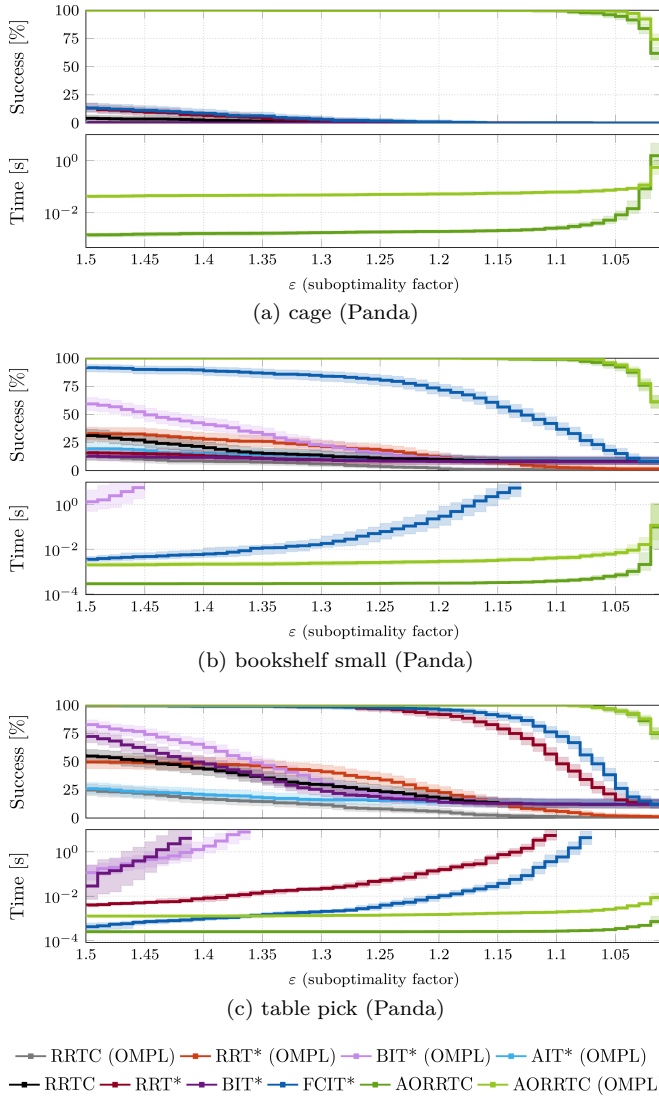


Fig. 4: 关于在 7 自由度 Panda 上 5 次试验中所有规划器的近似最优结果，用于 MotionBenchMaker 数据集中一个来自笼子 (a)、小书架 (b) 和桌子拾取 (c) 环境中的单一问题。顶部图表显示了在经验确定的最优解 c^* 以内的次优因子 ϵ 内找到解决方案的运行百分比，以及 Clopper-Pearson 99 % 置信区间。底部图表显示了在经验确定的最优解 c^* 以内的次优因子 ϵ 内找到解决方案的中位时间，以及非参数 99 % 置信区间。

行在 7 个 DoF Panda 机器人手臂 MBM 问题上，而只有 VAMP 规划器和 OMPL 版本的 AORRTC 和 RRT-Connect 在更难的 8 个 DoF Fetch MBM 问题上运行。OMPL 规划器没有在更难的 Fetch 问题上测试，因为在可用的规划时间内它们无法可靠地找到解决方案。所有测试均在 Ubuntu 22.04 的 Intel i7-9750H CPU 和 32GB RAM 上运行，规划算法用 C++17 实现。所有规划器使用默认的 VAMP 和 OMPL 采样器，每次试验的播种不同。OMPL 规划器使用 VAMP 的碰撞检测后台。规划器有 10 秒钟的时间来评估每个问题。所有图形的时间和成本轴为对数刻度。

给定环境中的所有问题的成功率如 Figs. 1 and 2 所示。Fig. 3 显示了每个规划器在一个问题上进行 100 次试验的成功率和中位解决方案成本，针对 Panda 和 Fetch 机器人手臂。

Fig. 4 展示了在 100 个问题上的 5 次试验中，每个规

划器在 Panda 机器人手臂的笼子、书架小型和桌子拾取环境中收敛到近似最优解的成功率和中位时间。这些结果比较了在哪些问题上找到了一个落在次优因数 ϵ 范围内的解决方案，与最优成本 c^* 的经验估计相比，以及找到满足该界限的解决方案的中位时间。给定问题的经验最优解取自开发和实验期间超过 700 次试验中发现的最低成本。

IV. 讨论

计算出的 AORRTC 在所有测试的 a.s.a.o. 规划器中表现最佳，并在所有环境中显著减少了找到初始解的时间 (Table I)。没有一个测试的 a.s.a.o. 规划器能够比 VAMP AORRTC 找到的解决方案收敛到更高质量的解 (Fig. 3)。OMPL AORRTC 优于所有 OMPL a.s.a.o. 和一些 VAMP a.s.a.o. 规划器，但 FCIT* 能够在某些环境中更快地找到解决方案 (Fig. 2)。这归功于 VAMP 的 SIMD 加速边缘评估的性能提升。唯一与 VAMP 和 OMPL 实现的 AORRTC 性能相似的规划器分别是 VAMP 和 OMPL 实现的 RRT-Connect。RRT-Connect 找到初始解的速度比 AORRTC 快微秒，但在给定额外规划时间的情况下无法改进此解。这是预期的，因为 RRT-Connect 的结果不包括路径简化的计算成本，而 AORRTC 找到的初始结果相当于 RRT-Connect 找到的简化解。OMPL 和 VAMP AORRTC 是唯一在所有 Panda 问题上找到解决方案的测试 a.s.a.o. 规划器。VAMP AORRTC 也是唯一在所有 Fetch 问题上找到解决方案的测试 a.s.a.o. 规划器。其他测试的 VAMP a.s.a.o. 规划器在遇到困难困难的 Fetch 问题时难以找到解决方案。VAMP RRT-Connect 也找到了所有测试问题的解决方案，但不是 a.s.a.o. 虽然 OMPL AORRTC 未能在允许时间内找到所有 Fetch 问题的解决方案，但它找到的 Fetch 问题解决方案明显更多，并且比除 AORRTC 和 RRT-Connect 以外的所有测试 VAMP 规划器更快找到这些问题的初始解。

OMPL 和 VAMP AORRTC 在显著更短的时间内收敛到比所有其他测试计划器更好的解决方案 (Fig. 4)。即使在困难的笼子问题上，AORRTC 的两个实现也能在毫秒级内可靠地收敛到接近经验最优的解决方案。AORRTC 的两个实现也比任何其他计划器用更少的时间收敛到这些解决方案。VAMP AORRTC 比 OMPL AORRTC 更快地找到给定的次优解，这得益于 VAMP 的 SIMD 加速边缘评估。

V. 结论

AORRTC 使用了 AO- x 元算法 [aox:1, aox:2] 和 RRT-Connect [rrtc] 的思想，设计了一种双向随时可用的 a.s.a.o. 算法，该算法可以快速找到初始解，随后逐渐收敛到最优解。

AORRTC 搜索一个 $(n+1)$ 维的增强搜索空间，其中前 n 维是配置空间， $(n+1)^{\text{th}}$ 维是其已到达的成本。它在这些增强搜索空间的序列上调用一个满意规划器，以当前最佳解的成本为界限，迭代地找到更高质量的解。这已被证明为 a.s.a.o. [aox:1, aox:2]。

AORRTC 在与 RRT-Connect 相同数量级上找到初始解，速度比其他 a.s.a.o. 规划算法快。然后，它使用剩余的规划时间以任意时间的方式找到比测试的其他 a.s.a.o. 算法更高质量的解。这在有和没有 SIMD 加速的情况下，在七个不同规划环境中的数百个问题中，对于 7 自由度

Panda 机器人手臂和 8 自由度 Fetch 机器人手臂得到了证明。

AORRTC 代表了一种不同的 a.s.a.o. 规划方法。大多数先前的 a.s.a.o. 算法通过提高精度来逼近连续值搜索空间，以便在增加规划时间的情况下找到更好的解决方案，并几乎必然地渐进收敛至最优解。而 AORRTC 则是快速从一组可行解中进行采样，这些解可能比当前解更优。这避免了维持高分辨率搜索空间近似的计算成本，使得 AORRTC 能够快速找到初始解并迅速收敛至最优

解。AORRTC 展示了这种 a.s.a.o. 规划替代方法的潜力，我们预期未来的研究将深入探索其全部影响。

未来的工作将研究进一步加速 AORRTC 收敛的方法。这可能包括扩大成本界限，以允许更差的中间解决方案，这些解决方案在简化后可能在成本界限内。其他工作将探讨不同距离函数的性能，并将 AORRTC 应用于优化除路径长度之外的成本函数，包括多变量成本函数。有关 AORRTC 实现的信息可在 <https://robotic-esp.com/code/aorrtc/> 找到。